Neocruncher 1.0 release
Benchmark Companion

Markov–Nekrasov Research Team

18 October 2026
Preliminary supplied observations; beta release companion

Abstract

This report separates empirical records from the main architecture paper. It presents four CPU scale observations from a supplied Alpha JSON summary, three native-hybrid wall times from a console transcript, and selected historical records. Completion, independent prefix comparison, complete-file identity, and internally reported enclosing intervals are distinguished. No controlled GPU speedup, general memory guarantee, or complete independent billion-digit verification is established.

1 Dataset, configuration, and evidence scope

The CPU observations use Chudnovsky pi, the CPU/native route, the existing CPU provider and 12 requested worker processes. The measured component version is `1.0.0a1`. Reported software includes Python 3.14.4, gmpy2 2.3.2, mpmath 1.4.1, NumPy 2.5.4 and PyOpenCL 2026.1.4. One observation is available at each reported size; no confidence intervals are established.

Only aggregated measurements and the conditions necessary to interpret them are published here. Raw development reports, execution identities, local paths, hardware fingerprints, complete-output digests and internal test procedures are excluded from this public companion. Detailed hardware models are not disclosed, which limits external reproducibility.

All CPU cases report completion with an independent comparison of the first 100,000 decimal places. The billion-place output was not available for complete independent inspection. New hybrid observations come from reported console wall times; their detailed phase and memory metadata remain unavailable for review. A measured development snapshot is not automatically a byte-identical qualification of the final release archive.

2 CPU scale results

Table 1: CPU/native observations. RSS is a sampled process-tree sum in GiB (2^{30} bytes), not an exact or guaranteed peak.

Decimal places	Calculation (s)	Process wall (s)	RSS (GiB)
1,000,000	0.438472	0.820471	0.372421
10,000,000	5.296766	5.699236	0.508232
100,000,000	77.555123	78.067759	1.683769
1,000,000,000	1174.762123	1176.479878	15.677929

Calculation time is the numerical implementation's reported interval. Process wall includes startup, calculation, output and the requested independent check. Their difference is not a direct measure of one particular overhead. At 1B, process wall is approximately 19 minutes 36.5 seconds. This is a reported completion time, not a performance record or a result portable to other hosts.

The status asserts independent agreement only over a 100,000-place prefix. The internal interval-based finalizer has a separate mathematical contract, conditional on correct arithmetic and bounds. Neither prefix agreement nor a stored digest by itself independently validates all billion decimal places.

3 Hybrid/native observations

Table 2: New hybrid console times versus CPU JSON process wall. The final column is descriptive, not a controlled backend-effect estimate.

Places	CPU wall (s)	Hybrid wall (s)	Hybrid / CPU
1M	0.820471	1.827	2.23
10M	5.699236	37.420	6.57
100M	78.067759	1929.621	24.72

All three console cases report `COMPLETED_PREFIX_CHECKED`. Their detailed verification metadata remains pending. The 100M wall time is approximately 32 minutes 9.6 seconds. There is no reported 500M completion in this dataset. The 10M and 100M results extend functional evidence for this route, while their wall times do not support describing it as faster than CPU execution.

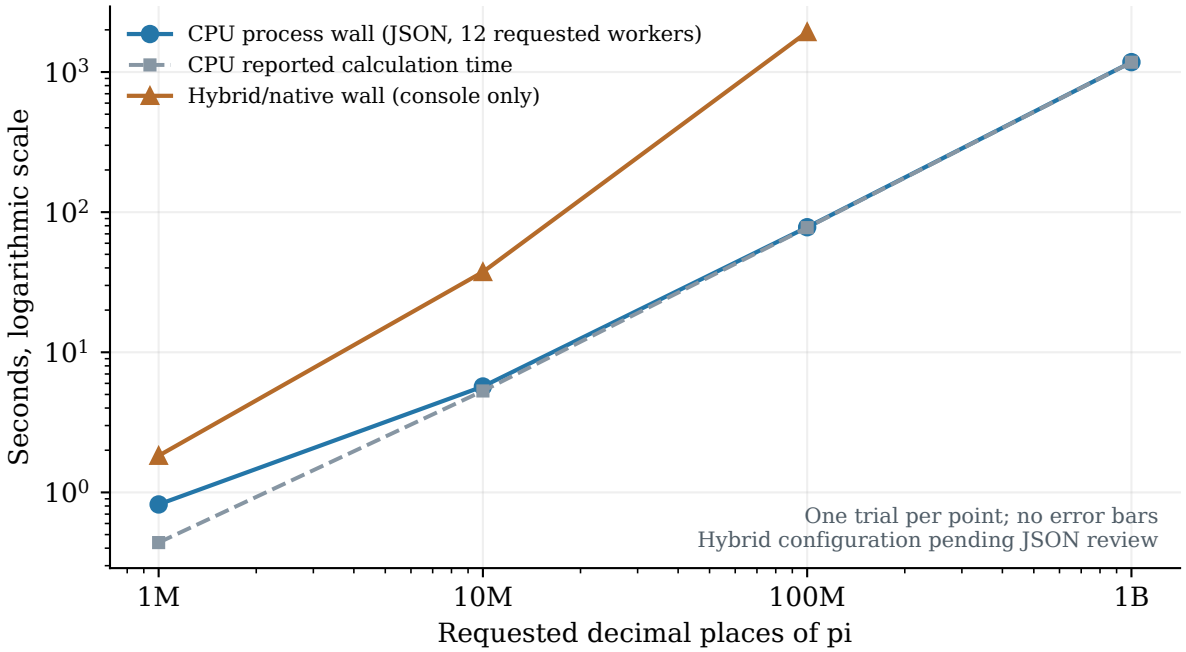


Figure 1: Supplied scale observations, on logarithmic axes. Lines connect measured points for readability and are not a fitted prediction. Hybrid data come from console text; CPU data come from a JSON summary. One trial per point.

4 CPU billion-place profile

The three top-level categories sum, subject to timer precision, to the reported 1174.762-second calculation interval: The binary-splitting interval contains parent merge time of 384.412479

Top-level category	Seconds	Share
Binary splitting, inclusive	544.404085	46.34%
Decimal conversion	235.888005	20.08%
Residual CPU / orchestration	394.470013	33.58%

seconds, ordered-result waiting of 159.477924 seconds, and submission timing of 0.055199 seconds. These are nested metrics; adding them again to the top-level categories would double-count work. Ordered waiting can include still-running subtree computation, transport and scheduling. It is not a CPU-idleness counter.

The parent merge time is about 32.72% of calculation time. It is a substantial observed region for future investigation, but its components are not all separately isolated. The residual likewise is not a measurement of pure Python overhead. It includes final arithmetic and other insufficiently decomposed work.

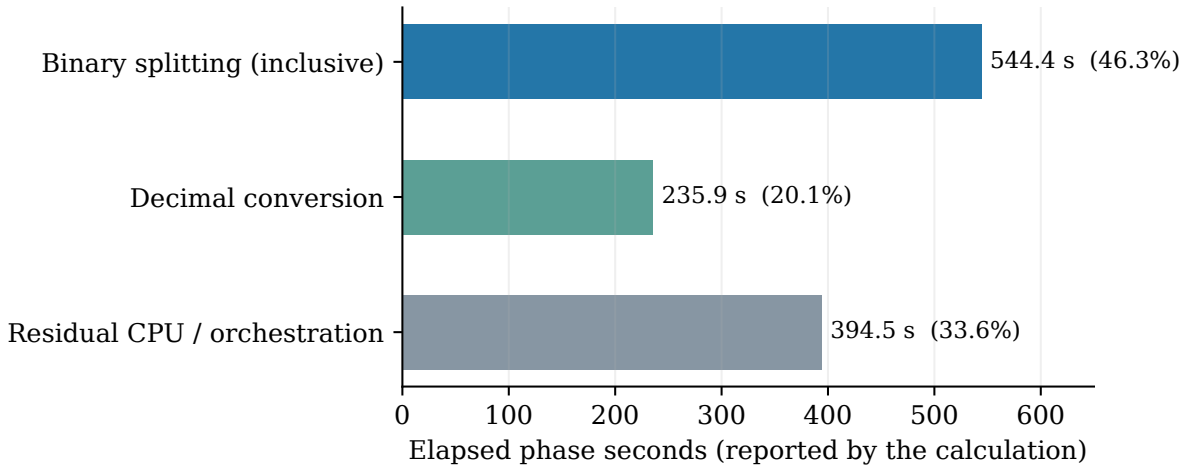


Figure 2: Reported top-level CPU calculation phases at 1B. Inclusive nested timers such as parent merging are deliberately not separate bars.

5 Memory and instrumentation

The CPU 1B report gives 16,834,048,000 bytes for the outer sampled process-tree RSS peak, 16,772,905,984 bytes for worker-tree PSS, and 16,769,781,760 bytes for main-process high-water RSS. These readings have different scopes and are not simultaneous components to add together. RSS sums can count shared pages more than once; PSS apportions them. High-water RSS from one process excludes descendants. Non-atomic sampling can miss short allocation spikes.

The supervisor records 2062 resource samples and 264.788646 seconds of summed elapsed sampling-call time at 1B, averaging approximately 0.1284 seconds per call. The field name `sampler_cpu_wall_cost_seconds` is misleading if interpreted as CPU usage: its implementation measures elapsed wall intervals. Sampling can overlap calculation. Subtracting 264.789 seconds from the runtime would not establish an uninstrumented time. The outer sampler adds another unisolated cost.

The job's planning interval is wider than the sampled peak and includes an upper warning. That does not mean the machine failed to recognize RAM, nor that the numerical engine allocated the estimate. Planning, sampled observation, and operating-system enforcement are different mechanisms. No claim of a calibrated universal memory formula is made.

6 Historical records and unsuccessful tests

A supplied 1.2 CPU/legacy record at 100M reports 96.271393 seconds of calculation, versus 77.555123 seconds in the Alpha summary. Both request 12 workers, and the output digests match. The numerical decrease is approximately 19.44%. This is a historical observation, not a replicated causal version-speedup estimate. Background activity, library details, instrumentation and source changes must be controlled before attributing the difference.

A supplied 1.0.1 CPU record at 10M reports 6.321697 seconds of calculation. A historical 1.2 hybrid/native 10M record reports 61.394909 seconds of calculation, whereas the new hybrid console reports 37.420 seconds of process wall. These are distinct timing scopes and source records; no version-speedup percentage is derived from that pair.

Earlier GPU-only native and Kolmogorov 10M attempts were reported to reach a 1200-second timeout during decimal conversion. They are historical failure reports, not complete new Alpha stress traces. The current data therefore neither establish a successful new 10M GPU-only run nor resolve the earlier failure. Hybrid success must not be substituted for GPU-only qualification.

7 Requirements for a stronger comparison

A later benchmark campaign should record device models, driver/ICD versions, CPU affinity, library builds, source fingerprints, verification settings and memory budgets. It should compare repeated cases under matching conditions, report variability, preserve failures internally, and separate instrumentation effects. For hybrid work it should record transfer bytes, kernel launches, device events, host phases and the actual subtree policy. Provider-specific PyKolm tests are required before claiming a PyKolm advantage.

The present report is intentionally descriptive. It supports the supplied completion records and identifies performance questions; it does not certify all digits, guarantee memory peaks, establish GPU acceleration or constitute a security audit. The companion architecture paper explains the numerical contracts and implementation boundaries needed to interpret these measurements.